

# РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

## ОТЧЕТ

### ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8

дисциплина:     Архитектура компьютера     \_

Студент: Ромеро Гаспар Фернандо Алексей

Группа:     НКАбд-02-25

МОСКВА

2025 г.



# Оглавление

Цель заботы

Теоретическое введение

2.1 Организация Стекa

2.2 Программирование циклов

2.3 Обработка аргументов командной строки

Выполнения лабораторной работы

3.2 Реализация циклов в NASM

3.3 Обработка аргументов командной строки

3.4 Арифметические операции с аргументами (Вычисление произведения)

Выполнение самостоятельной работы

Выводы

Список литературы

## **Цель работы**

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

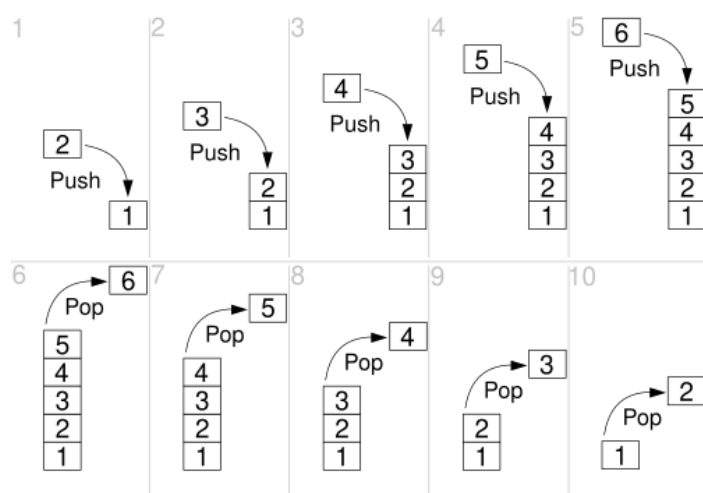
## Теоретическое введение

### 2.1 Организация Стека

Стек — это динамическая область памяти типа LIFO (Last-In, First-Out — «последним пришёл — первым ушёл»), которая хранит временные данные во время выполнения программ. Он организован как вектор, растущий в сторону уменьшения адресов памяти, и управляется процессором с помощью регистра ESP/RSP (указатель стека). Каждая вызванная функция создаёт «кадр стека» (stack frame), который содержит её локальные переменные, параметры и адрес возврата. Эта структура является фундаментальной для управления вызовами функций, рекурсии и хранения временных данных.

В языке ассемблера стек управляется с помощью специальных инструкций: PUSH (помещает данные и уменьшает указатель) и POP (извлекает данные и увеличивает указатель). Регистр EBP/RBP (указатель базы) обычно указывает на начало текущего кадра стека, обеспечивая структурированный доступ к локальным переменным и параметрам. Организация стека следует специфическим соглашениям архитектуры; в x86-64 с System V ABI первые 6 параметров передаются через регистры, в то время как дополнительные параметры и временные данные используют стек. Каждый кадр стека включает пространство для локальных переменных, сохраняемых регистров и адреса возврата.

Теперь переведи это: Стек не только управляет потоком программы, но и служит для хранения автоматических переменных. Его правильное управление предотвращает уязвимости, такие как переполнение буфера (buffer overflow), когда данные превышают выделенное пространство и могут перезаписать адрес возврата. Современные компиляторы реализуют средства защиты, такие как канарейки стека (stack canaries) и рандомизация адресного пространства (ASLR). В разработке систем понимание организации стека имеет решающее значение для отладки, оптимизации и программирования низкого уровня.



## 2.2 Программирование циклов

В 32-разрядной архитектуре x86 циклы реализуются с использованием 32-разрядных регистров, таких как EAX, EBX, ECX, EDX, ESI, EDI, EBP и ESP, для счётчиков и условий. Специализированная инструкция 'LOOP' автоматически уменьшает значение ECX и переходит к метке, если  $ECX \neq 0$ , однако она неэффективна на современных процессорах.

Архитектура x86-32 включает специализированные инструкции для управления циклами:

- **LOOP / LOOPZ / LOOPNZ:** Уменьшают значение ECX и выполняют переход на основе значения ECX и флагов.
- **JCXZ / JECXZ:** Выполняют переход, если  $ECX = 0$  (без уменьшения значения).
- **REP / REPE / REPNE:** Префиксы для строковых инструкций (MOVS, CMPS, SCAS, STOS).

| Инструкция           | Используемый регистр | Операция                        | Типичное использование                          |
|----------------------|----------------------|---------------------------------|-------------------------------------------------|
| <b>LOOP</b>          | ECX                  | DEC ECX; JNZ метка              | Простые циклы со счётчиком                      |
| <b>LOOPE/LOOPZ</b>   | ECX + ZF             | DEC ECX; JNZ метка<br>если ZF=1 | Циклы с дополнительным условием (пока равно)    |
| <b>LOOPNE/LOOPNZ</b> | ECX + ZF             | DEC ECX; JNZ метка<br>если ZF=0 | Циклы с дополнительным условием (пока не равно) |
| <b>JECXZ</b>         | ECX                  | Переход если ECX=0              | Быстрая проверка счётчика на ноль               |
| <b>REP</b>           | ECX                  | Повторяет инструкцию ECX раз    | Операции со строками/блоками                    |
| <b>DEC + JNZ</b>     | Любой                | <b>DEC регистр; JNZ метка</b>   | Циклы с произвольным регистром-счётчиком        |

|                      |          |                                  |                                              |
|----------------------|----------|----------------------------------|----------------------------------------------|
| <b>CMP + Jcc</b>     | Любой    | CMP регистр/память;<br>Jcc метка | Условные циклы<br>(while, for)               |
| <b>ADD/SUB + JMP</b> | Любой    | Арифметика;<br>метка             | JMP Циклы с сложной<br>логикой<br>инкремента |
| <b>REPE/REPZ</b>     | ECX + ZF | Повтор пока ECX≠0 и<br>ZF=1      | Поиск/сравнение<br>строк (пока равно)        |
| <b>REPNE/REPNZ</b>   | ECX + ZF | Повтор пока ECX≠0 и<br>ZF=0      | Поиск/сравнение<br>строк (пока не<br>равно)  |

## 2.3 Обработка аргументов командной строки

В Linux x86 аргументы командной строки передаются программе через стек при вызове функции `main()`. Согласно соглашению, в начале выполнения стек содержит, в порядке возрастания адресов: `argc` (счётчик аргументов), `argv` (вектор указателей на строки аргументов) и `envp` (переменные окружения). Регистр ESP указывает на `argc`. Например, для программы, запущенной как `./programa arg1 arg2`, начальное содержимое стека будет следующим: `3` (`argc`), адрес `"/programa"`, адрес `"arg1"`, адрес `"arg2"`, `NULL`, за которым следуют переменные окружения.

В ассемблере NASM для x86-32 доступ к аргументам осуществляется с помощью прямой адресации в стеке. В начале `_start` или `main`

Всегда следует проверять `argc` перед доступом к `argv[n]`. Доступ к `argv[argc]` возвращает `NULL` (0). Linux ограничивает общую длину аргументов значением `MAX_ARG_STRLEN` (131072 байт) и общее их количество значением `MAX_ARG` (2097152), однако на практике необходимо проверять каждый доступ к памяти.

## Выполнения лабораторной работы

### 3.1 Реализация циклов в NASM

Напишите программу `lab8-1.asm`, чтобы продемонстрировать, как работают циклы и насколько важно сохранять контекст регистра `ecx`.

При написании программы вы столкнетесь с проблемой: при изменении регистра `ecx` внутри тела цикла (например, путем его уменьшения с помощью инструкции ``sub ecx, 1``) команда цикла работает некорректно, поскольку она также использует `ecx` в качестве счетчика.

Для обеспечения надежности цикла используйте стек:

1. Перед изменением: ``push ecx`` (сохранение текущего счетчика).
2. После выполнения операций: ``pop ecx`` (восстановление счетчика). Это позволяет выполнять внутренние операции над `ecx` без прерывания логики итерации.

```
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 14
14
13
12
11
10
9
8
7
6
5
4
3
2
1
```

```
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nano lab8-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 14
13
11
9
7
5
3
1
```

```
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nano lab8-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 14
13
12
11
10
9
8
7
6
5
4
3
2
1
0
```

### 3.2 Обработка аргументов командной строки

Напишите программу `lab8-2.asm`, которая демонстрирует, как получить и вывести все



аргументы, переданные ей в начале. Сначала программа извлекает из стека количество аргументов (`argc`) и имя программы (`argv[0]`). Затем в цикле она последовательно извлекает и выводит оставшиеся аргументы.

```
fernando@ubuntu25-10:~/work/arch-pc/lab08$ touch lab8-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nano lab8-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nasm -f elf lab8-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-2 lab8-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ./lab8-2 11 21 'Chicken'
11
21
Chicken
```

### 3.3 Арифметические операции с аргументами (Вычисление произведения)

Для решения этой задачи необходимо изменить программу, вычисляющую сумму своих аргументов, на программу, вычисляющую их произведение.

Для этого вносятся следующие ключевые изменения:

- Инициализируется регистр результата (`esi`) значением 1 (единица умножения).
- Команда ``add`` заменяется командой ``imul`` (целочисленное умножение).

```
fernando@ubuntu25-10:~/work/arch-pc/lab08$ touch lab8-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nano lab8-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ./lab8-3 11 21 23 29 15
Результат: 99
```

## Выполнение самостоятельной работы

1. Напишите программу, которая находит сумму значений функции  $f(x)$  для  $x = x_1, x_2, \dots, x_n$ , т.е. программа должна выводить значение  $f(x_1) + f(x_2) + \dots + f(x_n)$ . Значения  $x_i$  передаются как аргументы. Вид функции  $f(x)$  выбрать из таблицы 8.1 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу на нескольких наборах  $x = x_1, x_2, \dots, x_n$ .

```
fernando@ubuntu25-10:~/work/arch-pc/lab08$ touch lab8-4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nano lab8-4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ nasm -f elf lab8-4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-4 lab8-4.o
fernando@ubuntu25-10:~/work/arch-pc/lab08$ ./lab8-4 1 2 3
Результат суммы  $f(x) = 15x + 2$ : 96
```

## **Выводы**

В ходе лабораторной работы были успешно реализованы и отлажены четыре программы на языке ассемблера NASM. Были освоены следующие ключевые навыки и понятия:

1. Зацикливание программы с использованием команды цикла и правильное управление регистром счетчика ехх через стек (push/pop).
2. Обработка аргументов командной строки через стек, включая получение количества аргументов (argc) и самих аргументов (argv).
3. Они выполняли арифметические действия с числовыми аргументами команд строки после их преобразования из строкового формата.

Все поставленные задачи выполнены

## Список литературы

1. GeeksforGeeks. (2022, 15 de noviembre). *Stack in Data Structure*.  
<https://www.geeksforgeeks.org/stack-data-structure/>
2. Microsoft Docs. (2023). *x64 stack usage*. <https://docs.microsoft.com/en-us/cpp/build/stack-usage>
3. OSDev Wiki. (2023). *Calling Conventions*. [https://wiki.osdev.org/Calling\\_Conventions](https://wiki.osdev.org/Calling_Conventions)
4. Intel Corporation. (2023). *\*Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2: Instruction Set Reference\**.  
<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
5. GeeksforGeeks. (2021, 20 de septiembre). *Loops in Assembly Language*.  
<https://www.geeksforgeeks.org/loops-assembly-language/>
6. Felix Cloutier. (2023). *x86 and amd64 Instruction Reference*.  
<https://www.felixcloutier.com/x86/>
7. GeeksforGeeks. (2023). *Command Line Arguments in C/C++*.  
<https://www.geeksforgeeks.org/command-line-arguments-in-c-cpp/>
8. Stack Overflow. (2022). *How to access command line arguments in assembly?*  
<https://stackoverflow.com/questions/1396122/how-to-access-command-line-arguments-in-assembly>
- 9.